3. Front-end com react

Sumário

3. Front-end com react 1
Módulo 1 - Introdução ao react 2
- O que é React? 2
- Componentização 2
- DOM Virtual 2

Módulo 1 - Introdução ao react

- **O que é React?**

React é um framework JavaScript criado pelo Facebook (atual Meta) que é usado para criar interfaces de usuário (UI) em aplicativos web.

Ele é popular por ser fácil de usar, altamente flexível e escalável, e é usado por muitas empresas de tecnologia, incluindo o Facebook, Instagram e Airbnb.

- **Componentização**

Uma das principais características do React é a componentização. Podemos pensar nos componentes como peças de uma máquina, que podem ser reaproveitadas por outros componentes e combinadas entre si. É por meio dos componentes que as páginas web são construídas no React.

Imagine que, em diferentes partes da sua aplicação, você precise utilizar um botão com as mesmas características visuais e comportamentais. Reescrever esse mesmo botão em vários lugares seria trabalhoso e pouco eficiente. É nesse contexto que a componentização se torna essencial.

Com a componentização, você pode criar um componente de botão uma única vez e reutilizá-lo sempre que precisar. Assim, sempre que quiser um botão com aquelas mesmas características, basta utilizar o componente, em vez de recriá-lo do zero.

- **DOM Virtual**

Outro conceito muito importante no React é o *Virtual DOM*. Ele é um processo pelo qual o React atualiza a aparência da página para o usuário. Antes de explicarmos esse conceito, é importante entender o que é o DOM.

O DOM **(***Document Object Model***)** é uma interface de programação para documentos HTML, XML e SVG. Ele fornece uma representação estruturada do documento na forma de uma árvore.

Para entendermos melhor, vamos a um exemplo. Observe o código da Imagem 1. Nele, vemos um documento HTML com as tags e informações da página. Na árvore DOM, cada tag é representada como um nó, ou seja, `html` é um nó, `head` é um nó, `div` é um nó e assim por diante.

Essa árvore é utilizada pelo JavaScript para tornar a página interativa, pois, ao manipular o DOM, o JavaScript altera o comportamento da página por meio de funções. Essas funções podem criar novos elementos no DOM, como uma nova tag

de parágrafo ou `div`, remover alguma tag existente ou alterar informações das tags, por exemplo, modificar o texto do parágrafo de “Para entender ...” para “O DOM é responsável por ...”, entre outras possibilidades.

Ou seja, o JavaScript utiliza o DOM para manipular a estrutura e o conteúdo da página.

Na Imagem 2, vemos a árvore DOM correspondente ao código da Imagem 1. Nela, podemos observar que cada tag do documento é representada como um nó. A árvore começa pelo nó raiz, o `document`, seguido pelo seu nó filho `html`, que possui dois nós filhos, `head` e `body`, e assim por diante. Note que os nós filhos no DOM correspondem às tags que estão contidas dentro de outras tags no código HTML.

```
arvore_dom.html ×
arvore_dom.html > html
1  <!DOCTYPE html>
2  <html lang="en">
3      <head>
4          <title>DOM</title>
5      </head>
6      <body>
7          <div>
8              <h1>Explicando a árvore DOM</h1>
9              <p>Para entender ...</p>
10         </div>
11     </body>
12 </html>
```

Imagem 1: Código html

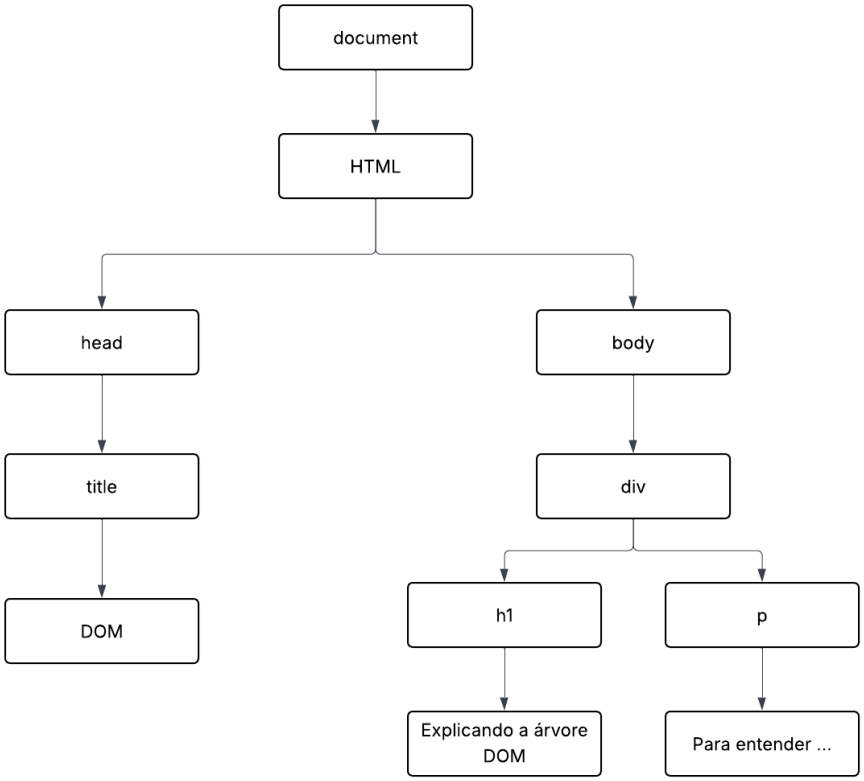

Imagem 2: Árvore dom do código

Agora que já sabemos o que é o DOM e qual a sua função, podemos entender o que é o *Virtual DOM*.

O Virtual DOM é utilizado pelo React no processo de atualização da página. Pense da seguinte forma: temos o DOM real, e o React cria uma representação virtual dele em memória. Sempre que ocorre uma alteração de estado na aplicação, o React cria uma nova versão dessa árvore virtual contendo as mudanças de estado.

Em seguida, o React compara a versão anterior do Virtual DOM com essa nova versão para identificar quais foram as diferenças entre as duas árvores. Depois de descobrir qual foi a alteração e em qual parte da árvore ela ocorreu, o React aplica apenas essas mudanças necessárias no DOM real, fazendo com que a página reflita o estado atual da aplicação.

Vale ressaltar que o Virtual DOM existe apenas em memória, como estruturas de dados em JavaScript. As mudanças só são efetivamente refletidas para o usuário quando o DOM real é atualizado.

A Imagem 3 ilustra o processo de *diffing*, no qual o React analisa as diferenças entre os nós de duas árvores do Virtual DOM. A árvore à esquerda representa a

cópia da interface no estado anterior, enquanto a árvore à direita representa a interface com a mudança de estado aplicada.

Após identificar quais alterações ocorreram e em quais nós da árvore, o React atualiza o DOM real aplicando apenas essas mudanças. As demais partes do DOM real permanecem inalteradas. Esse processo de atualização é ilustrado na Imagem 4 e é conhecido como *reconciliation*.

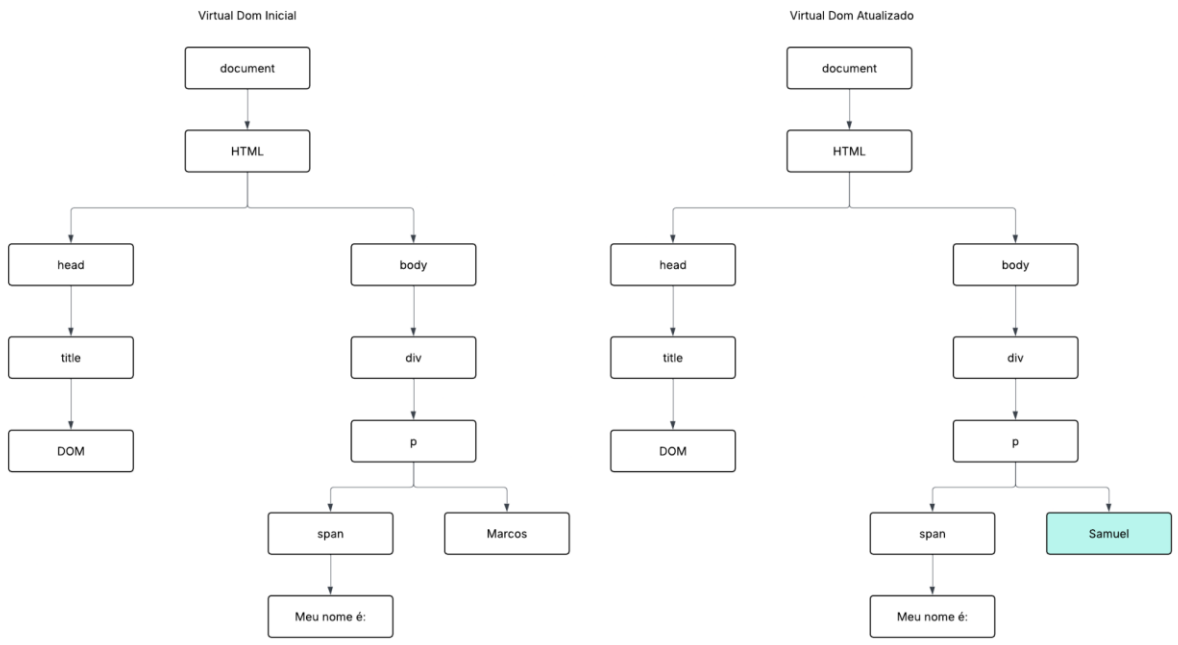

Imagem 3: Processo de diffing no Virtual DOM

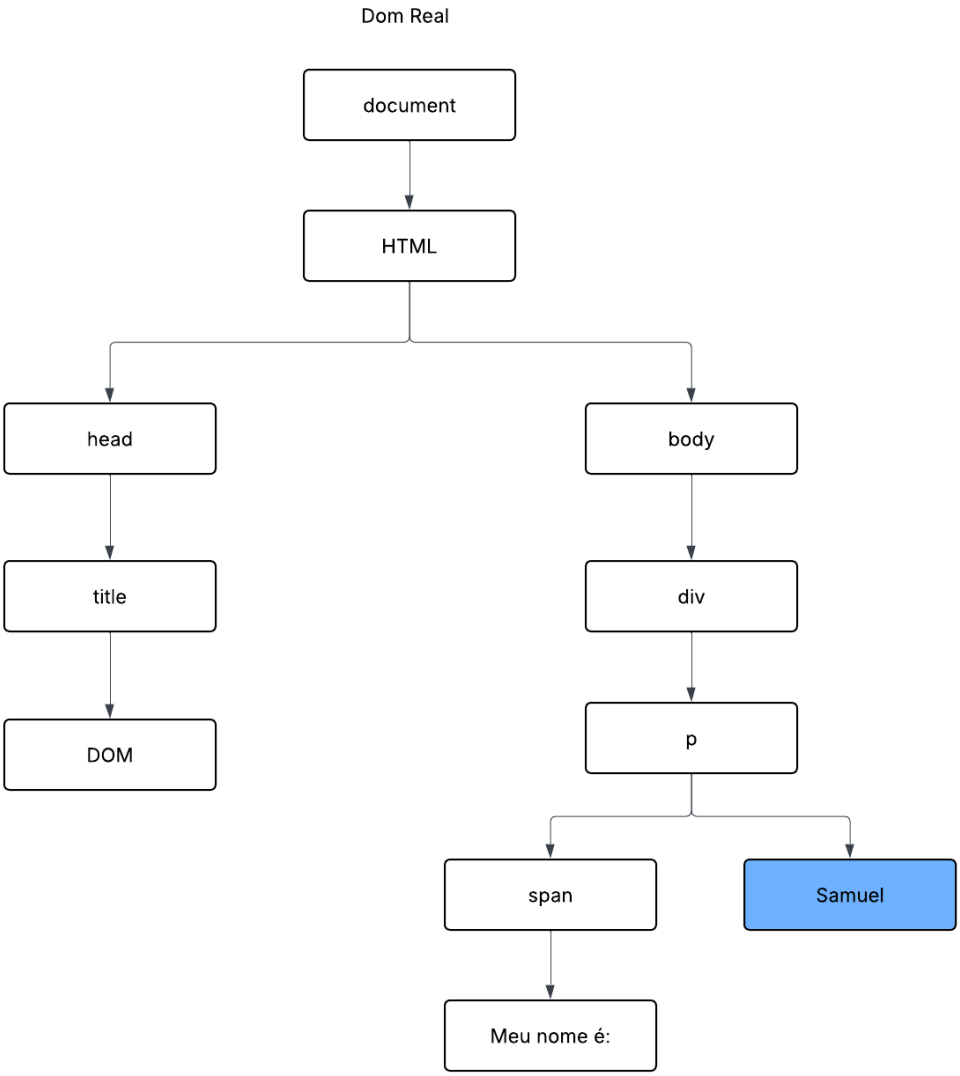

Imagem 4: DOM real com as alterações (reconciliation)